

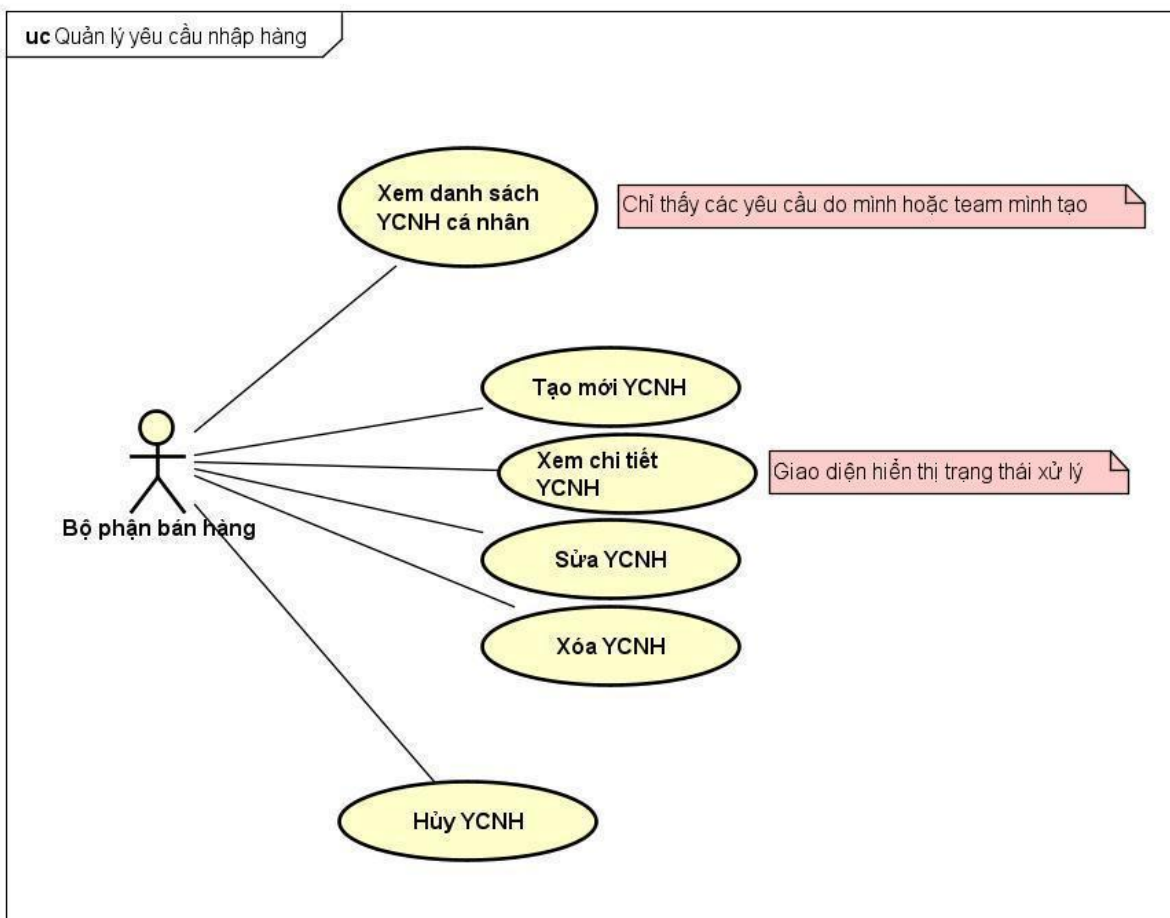
HỆ THỐNG PHẦN MỀM ĐẶT HÀNG NHẬP KHẨU

PHÂN TÍCH YÊU CẦU

Nhóm 8:

Họ và tên	MSSV	Usecase
Nguyễn Hoàng Thanh Tú	20235858	Tổng hợp/xem danh sách yêu cầu nhập hàng

1. Biểu đồ use case phân rã



2. Đặc tả Use case

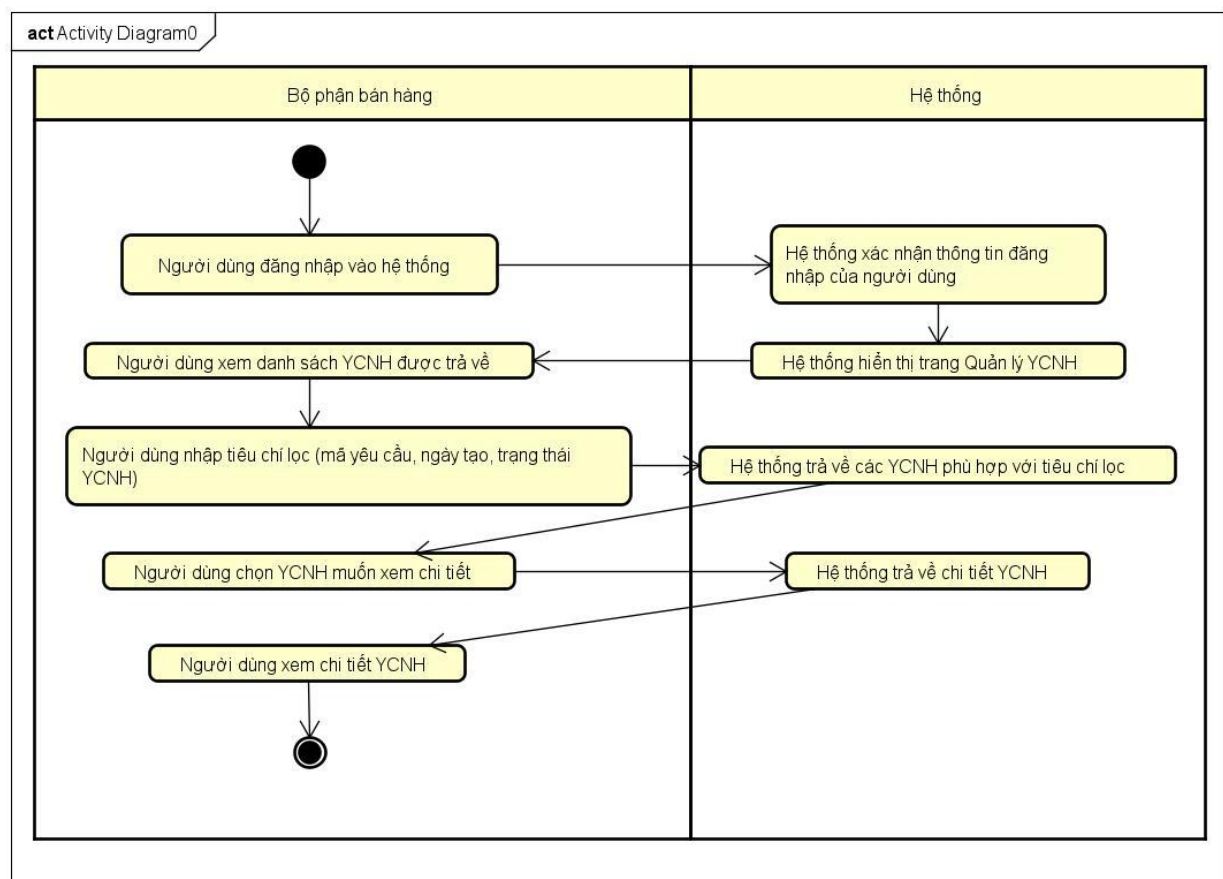
Usecase “Tổng hợp/Xem danh sách YCNH” - Tú

Mã Use case	UC001	Tên Use case	Đăng nhập
Tác nhân	Bộ phận bán hàng		
Tiền điều kiện	Người dùng đã đăng nhập vào hệ thống		
Luồng sự kiện chính (Thành công)	STT	Thực hiện bởi	Hành động
	1.	Bộ phận bán hàng	Truy cập vào trang quản lý YCNH
	2.	Hệ thống	Xác thực thông tin đăng nhập và hiển thị danh sách YCNH
	3.	Bộ phận bán hàng	Nhập các tiêu chí lọc (mã YCNH, ngày tạo, trạng thái)
	4.	Hệ thống	Trả về và hiển thị danh sách YCNH phù hợp với tiêu chí lọc
	5.	Bộ phận bán hàng	Chọn một YCNH để xem chi tiết
	6.	Hệ thống	Trả về thông tin chi tiết YCNH được chọn
Luồng sự kiện thay thế	STT	Thực hiện bởi	Hành động
	4a	Hệ thống	Thông báo không tìm thấy yêu cầu phù hợp nếu không có kết quả
Hậu điều kiện	Không		

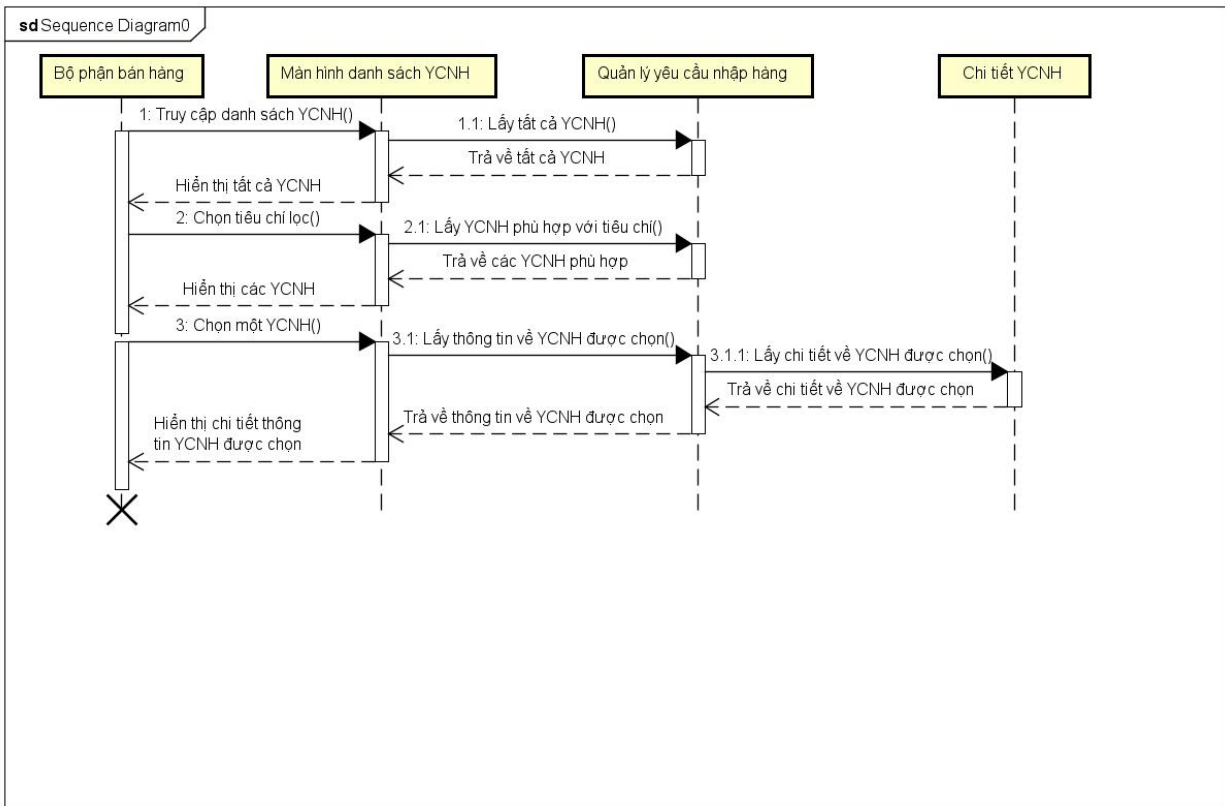
Dữ liệu đầu vào

STT	Trường dữ liệu	Mô tả	Bắt buộc?	Ví dụ
1	Mã YCNH	Mã định danh yêu cầu	Không	REQ001
2	Ngày tạo	Ngày tạo YCNH	Không	06/06/2026

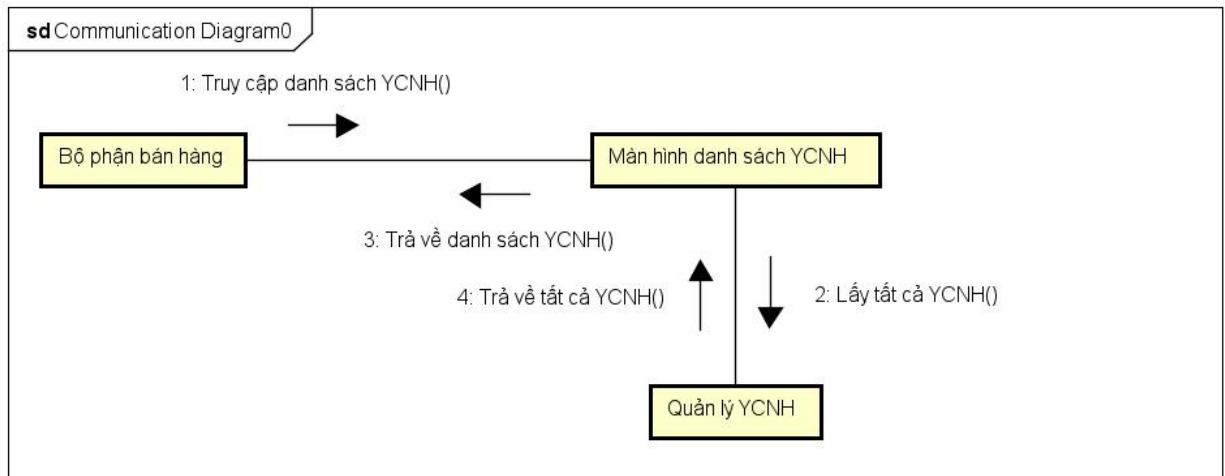
3. Activity Diagram



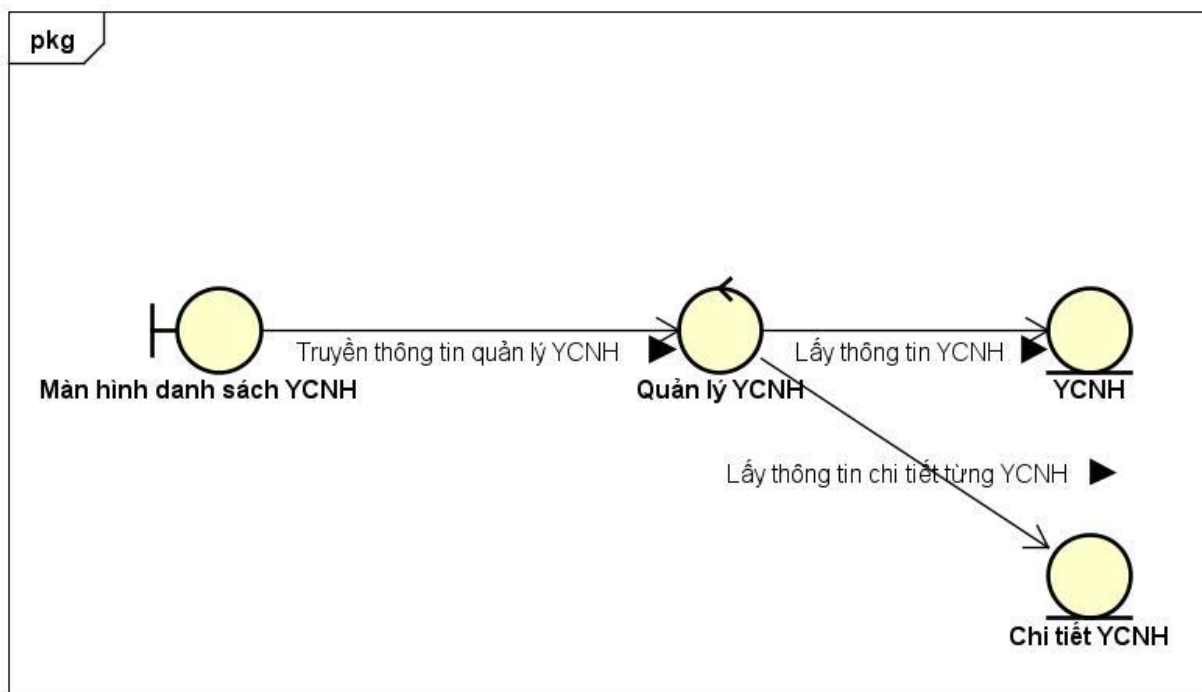
4. Sequence Diagram (mức phân tích)



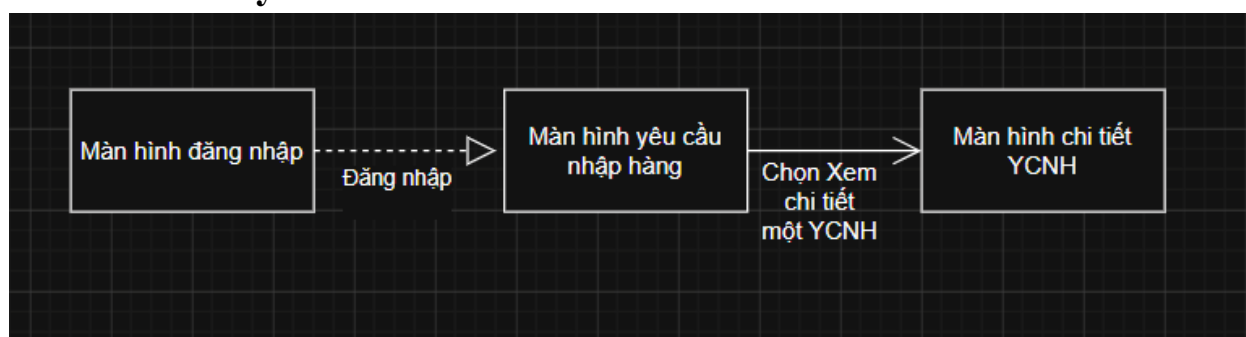
5. Communication Diagram



6. Biểu đồ lớp (mức phân tích)

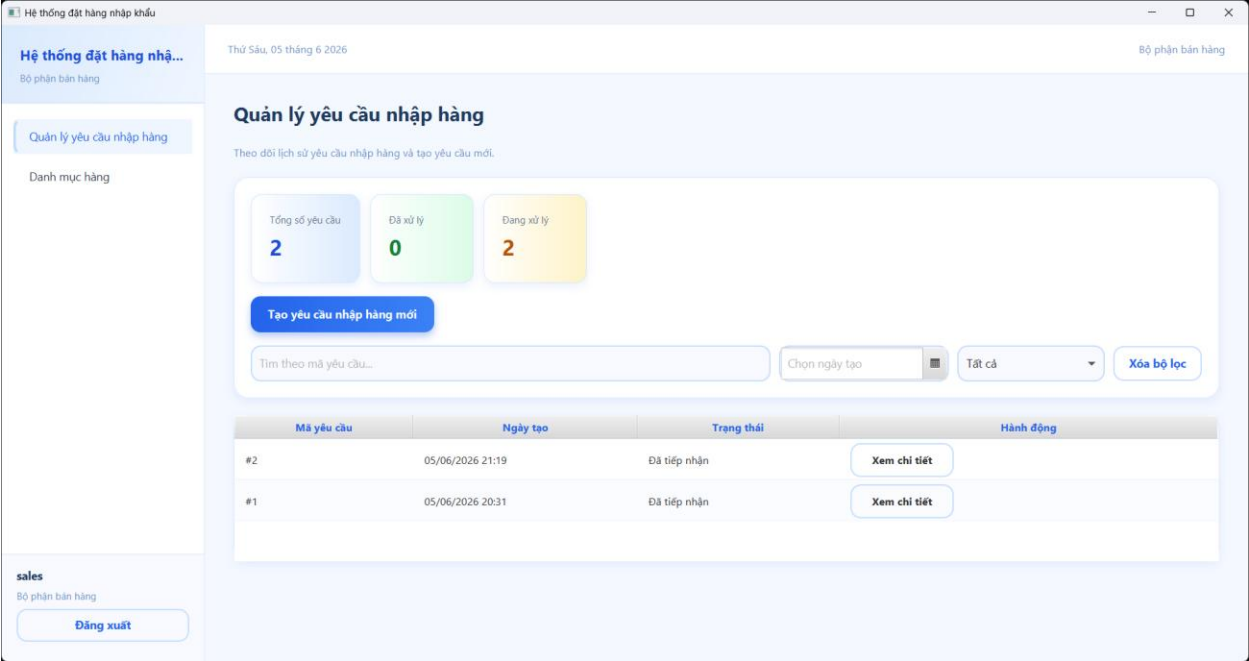


7. Sơ đồ chuyển màn hình



8. Đặc tả màn hình

8.1. Màn hình Quản lý yêu cầu nhập hàng



Tên thành phần	Hành động	Chức năng / Hành vi
Tiêu đề trang	Xem	Hiển thị tên chức năng (VD: "Quản lý yêu cầu nhập hàng") để người dùng nhận biết màn hình đang thao tác.
Bảng danh sách YCNH	Xem	Hiển thị danh sách các yêu cầu nhập hàng. Bao gồm các cột thông tin: Mã yêu cầu, Ngày tạo, Trạng thái (Đã tiếp nhận, Đã hoàn thành, Đã hủy, Đang xử lý).

Ô nhập Tìm theo mã yêu cầu	Nhập số	Cho phép người dùng nhập mã yêu cầu để tìm yêu cầu nhập hàng tương ứng
Ô lựa chọn Chọn ngày tạo	Nhập ngày tháng hợp lệ hoặc chọn ngày tháng từ bảng	Cho phép người dùng nhập ngày tháng hợp lệ hoặc chọn từ bảng để tìm yêu cầu nhập hàng tương ứng (các ngày tháng không hợp lệ sẽ không có tác dụng)
Ô lựa chọn Trạng thái	Chọn trạng thái từ danh sách drop down	Cho phép người dùng chọn trạng thái đơn hàng từ danh sách drop down để tìm yêu cầu nhập hàng tương ứng

Nút "Xem chi tiết" (Trên từng dòng)	Click	Chuyển đến màn hình chi tiết của yêu cầu nhập hàng tương ứng.
--	--------------	---

8.2. Xem chi tiết yêu cầu nhập hàng

Hệ thống đặt hàng nhập khẩu

Hệ thống đặt hàng nhập...

Bộ phận bán hàng

Quản lý yêu cầu nhập hàng

Danh mục hàng

sales

Bộ phận bán hàng

Đăng xuất

Thứ Sáu, 05 tháng 6 2026

Bộ phận bán hàng

Chi tiết yêu cầu nhập hàng

Xem toàn bộ thông tin của yêu cầu nhập hàng đã tạo.

Quay lại danh sách

Yêu cầu nhập hàng #1

Ngày tạo: 05/06/2026 20:31

Trạng thái: Đã tiếp nhận

Các mặt hàng cần nhập

P001 - Sữa bột nhập khẩu

Số lượng: 10

Đơn vị: hộp

Ngày mong muốn: 2026-06-12

P002 - Bánh quy

Số lượng: 5

Đơn vị: thùng

Ngày mong muốn: 2026-06-12

Thông tin hủy yêu cầu

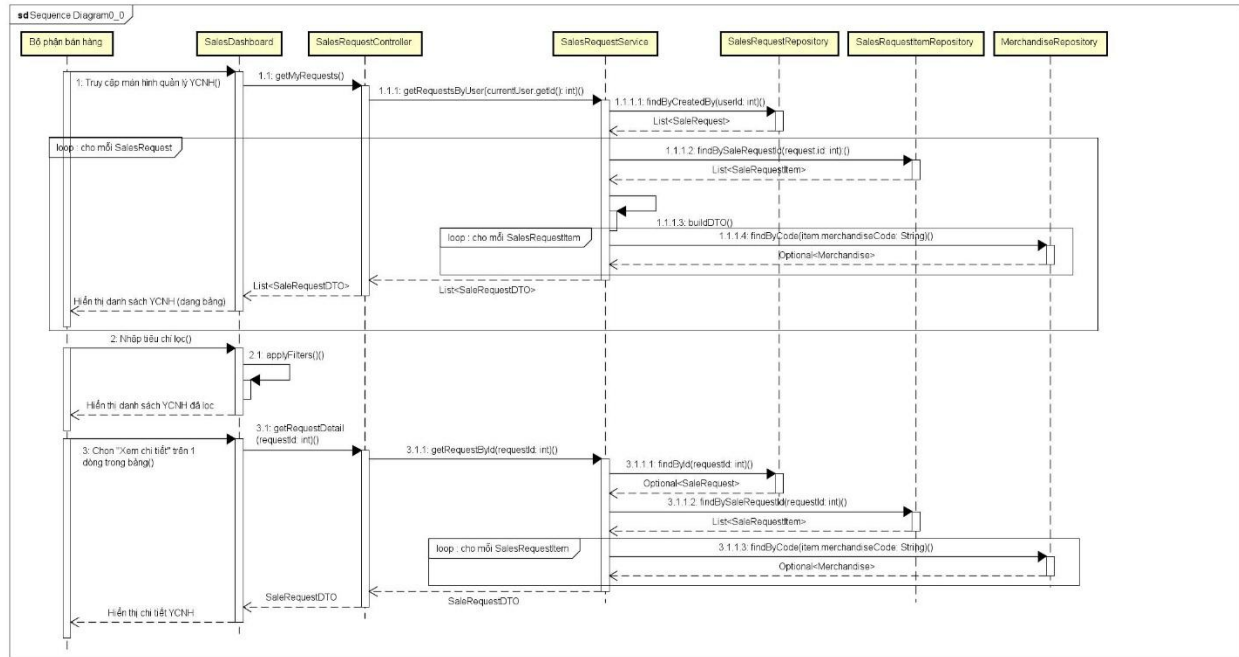
Gửi yêu cầu hủy

Báo cáo từ bộ phận kho

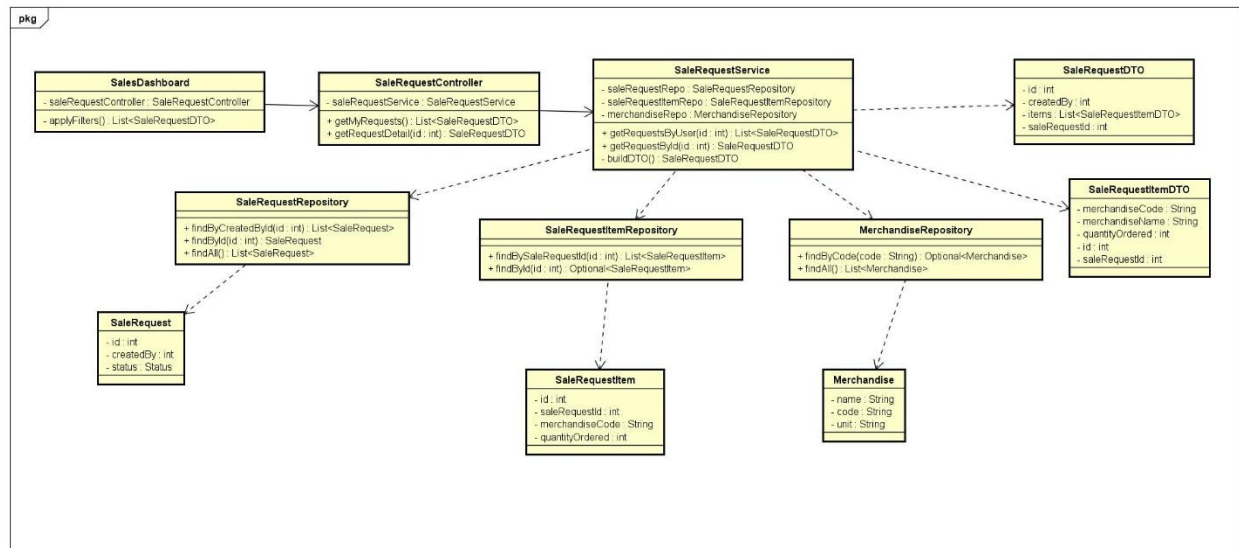
Chưa có đơn đặt hàng nào được gửi tới site.

Tên thành phần	Hành động	Chức năng / Hành vi
Tiêu đề trang	Xem	Hiển thị tên chức năng
Tiêu đề YCNH	Xem	Hiển thị mã số của yêu cầu nhập hàng đang xem
Khối thông tin tĩnh (Các mặt hàng cần nhập, số lượng và đơn vị của mỗi loại, ngày mong muốn nhận)	Xem	Hiển thị các thông tin dạng chỉ đọc (Read-only) được hệ thống tự động lấy từ dòng dữ liệu yêu cầu nhập hàng mà người dùng vừa chọn.

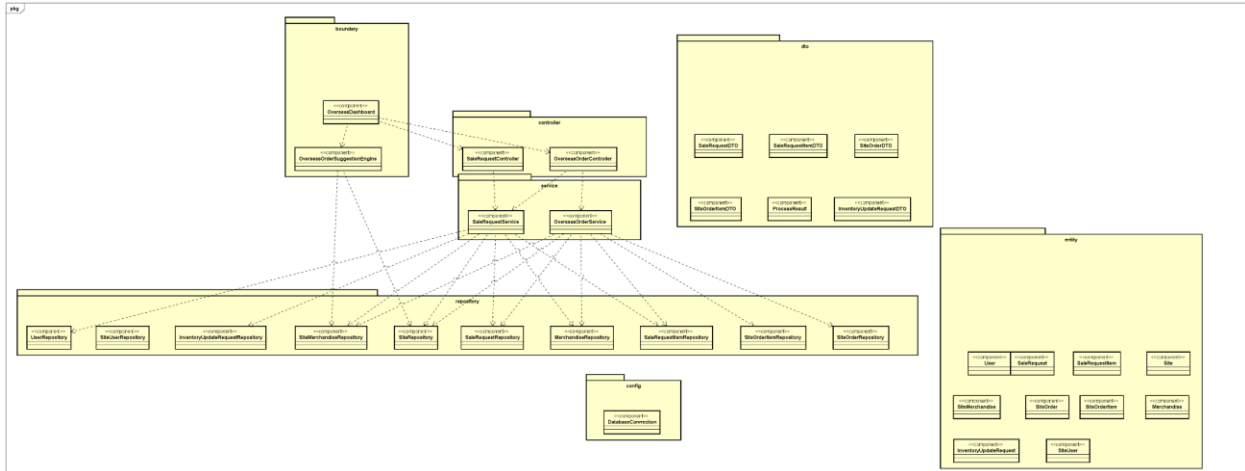
9. Biểu đồ trình tự (mức thiết kế)



10. Biểu đồ lớp (mức thiết kế)



11. Biểu đồ phụ thuộc gói



12. Áp dụng Mẫu thiết kế (Design Patterns)

12.1. Mẫu thiết kế DTO (Data Transfer Object)

- **Thiết kế ban đầu (Chưa chỉnh sửa):** Truyền trực tiếp các đối tượng thực thể cơ sở dữ liệu (SaleRequest, SaleRequestItem) từ tầng Repository lên thẳng tầng Giao diện (SalesDashboard) để hiển thị lên bảng. Điều này gây ra các vấn đề: lộ cấu trúc bảng SQL ra Frontend, truyền nhiều dữ liệu không cần thiết (ví dụ: khi hiển thị danh sách, chỉ cần mã yêu cầu, ngày tạo, trạng thái; không cần các trường như cancel_request_reason hay cancel_resolution_note), và khó thay đổi cấu trúc hiển thị mà không ảnh hưởng đến tầng Entity.
- **Cách chỉnh sửa áp dụng Pattern:** Khởi tạo các lớp DTO (Data Transfer Object) chuyên dụng. SaleRequestDTO dùng cho danh sách yêu cầu, chỉ chứa các trường cần thiết như id, ngày tạo, trạng thái và tên người tạo. SaleRequestItemDTO dùng cho chi tiết yêu cầu, chứa mã hàng, tên hàng, số lượng, đơn vị và ngày nhận mong muốn. Tầng Service có nhiệm vụ "chuyển đổi" (mapping) dữ liệu từ Entity sang DTO trước khi trả về cho Controller. Controller sau đó chỉ việc chuyển tiếp DTO lên Giao diện để hiển thị. Quá trình chuyển đổi này cũng là cơ hội để "làm giàu" dữ liệu, ví dụ: lấy thêm tên người tạo từ UserRepository và tên hàng từ MerchandiseRepository để gắn vào DTO.

- **Ý nghĩa đem lại:**
 - + **Bảo mật cấu trúc cơ sở dữ liệu:** Các bảng SQL không bị lộ cấu trúc lên Frontend.
 - + **Tối ưu băng thông:** DTO chỉ chứa những trường dữ liệu thực sự cần thiết để hiển thị lên giao diện, loại bỏ các trường dư thừa từ database.
 - + **Linh hoạt trong hiển thị:** DTO có thể chứa dữ liệu "giàu" hơn Entity bằng cách kết hợp thông tin từ nhiều nguồn khác nhau.

12.2. Mẫu thiết kế Repository (Collection)

- **Thiết kế ban đầu (Chưa chỉnh sửa):** Logic truy xuất cơ sở dữ liệu (câu lệnh SQL, xử lý Connection, ResultSet) được viết trực tiếp bên trong Service. Điều này khiến Service vừa phải xử lý nghiệp vụ, vừa phải xử lý truy vấn SQL, dẫn đến vi phạm nguyên tắc Đơn trách nhiệm và làm cho code trở nên khó bảo trì.
- **Cách chỉnh sửa áp dụng Pattern:** Tách toàn bộ logic truy xuất dữ liệu vào các lớp Repository riêng biệt. SaleRequestRepository chịu trách nhiệm truy vấn bảng sale_requests, SaleRequestItemRepository chịu trách nhiệm truy vấn bảng sale_request_items, MerchandiseRepository chịu trách nhiệm truy vấn bảng merchandises, và UserRepository chịu trách nhiệm truy vấn bảng users. Service bây giờ chỉ việc gọi các phương thức từ Repository và không cần biết bên trong Repository làm gì. Repository có nhiệm vụ kết nối database, thực thi câu lệnh SQL, map dữ liệu từ ResultSet sang Entity, và trả về Entity cho Service.
- **Ý nghĩa đem lại:**
 - + **Tuân thủ nguyên tắc Đơn trách nhiệm (SRP):** Mỗi lớp chỉ có một lý do để thay đổi. Service thay đổi khi nghiệp vụ thay đổi; Repository thay đổi khi cấu trúc database thay đổi.
 - + **Tái sử dụng code:** Các phương thức như tìm theo id, lấy tất cả có thể được dùng lại ở nhiều Service khác nhau.

+ **Dễ kiểm thử:** Có thể tạo Mock Repository để kiểm thử Service mà không cần database thật.

13. Áp dụng Nguyên lý thiết kế (SOLID Principles)

13.1. Nguyên lý Đơn trách nhiệm (SRP - Single Responsibility Principle)

- **Cách áp dụng trong Use case:** Mỗi lớp trong kiến trúc chỉ đảm nhiệm đúng MỘT lý do để thay đổi. Cụ thể:
 - + **SalesDashboard (Boundary):** Chỉ lo việc hiển thị giao diện JavaFX, bắt sự kiện click chuột từ người dùng, nhập liệu từ bàn phím, và hiển thị dữ liệu lên bảng. Nó hoàn toàn "mù" về nghiệp vụ cơ sở dữ liệu và logic xử lý.
 - + **SaleRequestController (Controller):** Chỉ lo việc nhận lệnh từ giao diện, kiểm tra quyền truy cập của người dùng đang đăng nhập (requireRole), và ủy quyền xử lý cho Service. Nó không biết dữ liệu được lấy từ đâu hay hiển thị như thế nào.
 - + **SaleRequestService (Service):** Chỉ chứa logic nghiệp vụ: lọc danh sách theo điều kiện, chuyển đổi Entity sang DTO, kiểm tra tính hợp lệ của dữ liệu. Nó không biết dữ liệu được hiển thị trên giao diện như thế nào và cũng không biết câu lệnh SQL được viết ra sao.
 - + **SaleRequestRepository (Repository):** Chỉ lo việc viết câu lệnh SQL, kết nối database, và map dữ liệu từ ResultSet sang Entity. Nó không chứa bất kỳ logic nghiệp vụ nào.
 - + **Entity classes (SaleRequest, SaleRequestItem, Merchandise, User):** Chỉ phản ánh cấu trúc dòng dữ liệu trong các bảng database. Chúng không chứa bất kỳ phương thức xử lý nghiệp vụ nào.
- **Ý nghĩa đem lại:**
 - + **Dễ bảo trì (Maintainable):** Khi muốn đổi giao diện từ JavaFX

sang Web, chỉ cần thay đổi tầng Boundary (SalesDashboard) và giữ nguyên toàn bộ tầng Controller + Service + Repository.

+ **Dễ gỡ lỗi (Debug):** Khi có lỗi xảy ra, có thể xác định chính xác tầng nào gây ra lỗi (lỗi SQL → vào Repository; lỗi hiển thị → vào Dashboard; lỗi quyền → vào Controller).

+ **Dễ mở rộng:** Thêm tính năng mới (ví dụ: xuất Excel danh sách yêu cầu nhập hàng) chỉ cần thêm method mới đúng chỗ mà không ảnh hưởng đến code cũ.

13.2. Nguyên lý Đảo ngược phụ thuộc (DIP - Dependency Inversion Principle)

- **Cách áp dụng trong Use case:** Các module cấp cao không được phụ thuộc vào các module cấp thấp; cả hai phải phụ thuộc vào abstraction (Interface). Trong luồng thiết kế hiện tại: Service (cấp cao) không khởi tạo trực tiếp các Repository. Thay vào đó, nó khai báo các Repository là attribute và các Repository này được "bơm vào" từ bên ngoài thông qua constructor (Dependency Injection). Điều này cho phép Service chỉ phụ thuộc vào "khái niệm" về Repository chứ không phụ thuộc vào cách cài đặt cụ thể. Để đúng hoàn toàn với DIP, cần tạo Interface cho mỗi Repository (ví dụ: ISaleRequestRepository, ISaleRequestItemRepository). Service sẽ chỉ phụ thuộc vào các Interface này, còn các class Repository cụ thể sẽ implement các Interface đó. Việc khởi tạo Repository nào được sử dụng sẽ được quyết định ở tầng cao hơn (ví dụ: ApplicationContext hoặc Dependency Injection framework).
- **Ý nghĩa đem lại:**
 - + **Dễ kiểm thử (Unit Testing):** Có thể dễ dàng tạo ra một Repository giả (Mock Object) để kiểm thử logic của Service mà không cần phải cắm database PostgreSQL thật. Điều này đặc biệt quan trọng để đáp ứng yêu cầu kiểm thử đơn vị.
 - + **Dễ thay đổi công nghệ:** Nếu muốn đổi từ PostgreSQL sang

MongoDB, chỉ cần viết implementation mới của Interface, không cần sửa bất kỳ dòng code nào trong Service.

+ **Giảm sự phụ thuộc (Loose Coupling):** Các lớp không còn phụ thuộc chặt chẽ vào nhau, dễ dàng thay thế các thành phần khi cần.

13.3. Nguyên lý Mở - Đóng (OCP - Open/Closed Principle)

- **Cách áp dụng trong Use case:** Các lớp nên "mở" cho việc mở rộng (extension) nhưng "đóng" cho việc sửa đổi (modification). Trong use case này, hệ thống lọc danh sách yêu cầu nhập hàng được thiết kế theo hướng mở để có thể thêm tiêu chí lọc mới mà không cần sửa code cũ. Hiện tại, hệ thống hỗ trợ ba bộ lọc: lọc theo mã yêu cầu, lọc theo ngày tạo, và lọc theo trạng thái. Mỗi bộ lọc là một method riêng biệt (matchesRequestCode, matchesCreatedDate, matchesStatus). Khi cần thêm bộ lọc mới (ví dụ: lọc theo người tạo, lọc theo mã hàng trong yêu cầu), chỉ cần viết thêm method mới và thêm một dòng vào pipeline lọc, mà không cần sửa các method lọc cũ. Về lâu dài, có thể thiết kế một hệ thống lọc tổng quát hơn bằng cách định nghĩa một Interface Filter, và mỗi tiêu chí lọc là một implementation riêng. Cách thiết kế này cho phép thêm bộ lọc mới mà hoàn toàn không cần sửa lại code xử lý lọc chính.
- **Ý nghĩa đem lại:**
 - + **Dễ mở rộng:** Khi nghiệp vụ phát sinh yêu cầu lọc mới, chỉ cần thêm code mới mà không sợ ảnh hưởng đến code cũ.
 - + **Giảm rủi ro:** Không lo làm hỏng tính năng đang hoạt động tốt khi thêm tính năng mới.
 - + **Tuân thủ nguyên tắc thiết kế tốt:** Code dễ đọc, dễ hiểu và dễ bảo trì

14. Mô tả module cần kiểm thử

14.1. Thông tin chung

Thuộc tính	Giá trị
Tên module	getRequestDetail
Lớp chứa	SaleRequestController
Kiểu trả về	SaleRequestDTO
Tham số đầu vào	requestId: int - mã số của yêu cầu nhập hàng cần xem chi tiết
Ngoại lệ có thể ném	IllegalStateException, IllegalArgumentException, SQLException

14.2. Mô tả chức năng

Phương thức này có nhiệm vụ trả về chi tiết của một yêu cầu nhập hàng dựa trên mã số requestId được cung cấp. Quá trình xử lý bao gồm:

1. Kiểm tra người dùng đã đăng nhập hay chưa
2. Kiểm tra người dùng có quyền SALES_DEPARTMENT không
3. Gọi xuống tầng Service để lấy dữ liệu
4. Kiểm tra quyền xem chi tiết
5. Trả về đối tượng SaleRequestDTO hoặc ném ngoại lệ.

14.3. Các rẽ nhánh điều kiện

Mã nhánh	Điều kiện	Kết quả
B1	currentUser == null	Ném IllegalStateException: "Chưa đăng nhập"
B2	currentUser.getRole() != SALES_DEPARTMENT	Cho phép xem
B3	currentUser.getRole() == SALES_DEPARTMENT	Kiểm tra thêm quyền

B4	saleRequestService.getRequestById () không tìm thấy	Ném IllegalArgumentException
B5	role == SALES_DEPARTMENT và createdBy != userId	Ném IllegalStateException
B6	Mọi điều kiện đều thỏa mãn	Trả về SaleRequestDTO

13. Mô tả module cần kiểm thử

13.1. Các test case thiết kế

Test Case ID	request Id	Trạng thái currentUser	Dữ liệu trong DB	Expected Output
TC-BLACK-01	1	null	Có request id=1	IllegalStateException
TC-BLACK-02	1	Role = OVERSEAS	Có request id=1	SaleRequestDTO
TC-BLACK-03	1	Role = ADMIN	Có request id=1	SaleRequestDTO
TC-BLACK-04	1	Role = SALES, id=100	createdBy=100	SaleRequestDTO
TC-BLACK-05	1	Role = SALES, id=200	createdBy=100	IllegalStateException
TC-BLACK-06	9999	Role = ADMIN	Không có	IllegalArgumentException

TC-BLACK-07	0	Role = ADMIN	Không có	IllegalArgumentException
TC-BLACK-08	-1	Role = ADMIN	Không có	IllegalArgumentException
TC-BLACK-09	1	Role = SALES, id=100	cancelRequested =true	SaleRequestDTO

14. Áp dụng kỹ thuật Kiểm thử hộp trắng

Test Case ID	requestId	currentUser	Mục tiêu nhân h	Expected Output
TC-WHITE-01	1	null	B1	IllegalStateException
TC-WHITE-02	1	Role=ADMIN	B2, B4, B6	SaleRequestDTO
TC-WHITE-05	1	Role=SALES, id=100	B3, B4, B6	SaleRequestDTO
TC-WHITE-06	1	Role=SALES, id=200	B3, B4, B5	IllegalStateException
TC-WHITE-07	9999	Role=ADMIN	B2, B4	IllegalArgumentException

15. Bảng tổng hợp test case

ID	Kỹ thuật	Mục tiêu	requestId	currentUser	Expected Output
TC-01	Hộp đen/trắng	Chưa đăng nhập	1	null	IllegalStateException
TC-02	Hộp đen/trắng	Quyền ADMIN	1	Admin	SaleRequestDTO
TC-03	Hộp đen/trắng	Quyền SALES (Đúng)	1	SALES_100	SaleRequestDTO
TC-04	Hộp đen/trắng	Quyền SALES (Sai)	1	SALES_200	IllegalStateException
TC-05	Hộp đen/trắng	Không tồn tại	9999	Admin	IllegalArgumentExcept ion